

Learning language-model RL from an official reproduction

We want to establish that our language-model reinforcement learning pipeline does what we think it does: generate answers, assign rewards, change weights, and evaluate those changed weights fairly. We start with the authors’ code for *Understanding R1-Zero-Like Training: A Critical Perspective* [1,2]. This is a small reproduction of its training method, with a smaller model workload and budget. It does not reproduce the paper’s published scores.

Why do this before recursive language-model RL?

In this experiment, the language model receives a mathematics question and writes one response. A checker compares its final answer with a reference. There are no live Python tools or recursive model calls in these responses. That makes the reward and update easier to inspect.

An RLM adds choices about decomposition, running code, reading intermediate results, and calling other models. A failure can then come from the language model, the surrounding software, the reward, or which actions receive credit. A working ordinary LLM experiment gives us a reference for the shared training machinery. It does not prove that those additional RLM decisions are trained correctly. We will still need separate checks of action boundaries, tool results, costs, and credit assignment.

Pretraining, SFT, and RL are different sources of learning

Stage	What supplies the learning target?
Pretraining	Predicting text from a large existing collection develops the starting language and mathematical abilities.
Supervised fine-tuning (SFT)	Supplied example responses tell the model which tokens to imitate for a prompt.
Reinforcement learning (RL)	The model samples its own responses. Rewards change how likely those responses are under its policy.

Our starting checkpoint is the **Qwen2.5-Math-1.5B base model** [3]. We add no SFT stage. “Base” does not mean untrained: its weights already contain substantial capabilities from earlier training. Calling this R1-Zero-like training does not mean beginning with random weights.

What has been established so far?

After training from the base model on 512 questions, the prescribed final checkpoint answered **80 of 128 fixed test questions correctly, versus 41 for the base model**. Forty-four answers became correct and five became incorrect. The trained model also reached the response limit much less often. This is a useful improvement in completed answers in this run. Page 4 explains the result and its limits; page 5 uses the smaller pilot to illustrate what an individual score change can mean.

Evidence cutoff: 7 October 2026, 10:17 UTC. Fixed 128-question comparison complete; expanded evaluations pending.

How eight answers produce an update

For each training question, the generator samples eight responses at temperature 1. The author reward function extracts the last boxed answer and checks it against the reference. A correct answer earns 1; a wrong or unparseable answer earns 0. There is no separate reward just for using a box. The training actor also assigns 0 to a response stopped by the token limit. The checker evaluates the answer, not the truth of every reasoning step [2].

A teaching example: two successes in a group of eight

Suppose one question produces rewards

$$r = (1, 0, 0, 1, 0, 0, 0, 0), \quad \bar{r} = 2/8 = 0.25.$$

Dr. GRPO subtracts this group’s mean from each reward:

$$A_i = r_i - \bar{r}, \quad A = (0.75, -0.25, -0.25, 0.75, -0.25, -0.25, -0.25, -0.25).$$

The two successful responses receive positive advantages; the other six receive negative advantages. The update pushes toward the successful sampled responses and away from the unsuccessful ones. If all eight answers are wrong, or all eight are correct, their advantages are all zero. The sum being zero in a mixed group does not cancel the learning signal: different responses have different token probabilities and gradients.

What is special about Dr. GRPO?

Dr. GRPO is the authors’ variant of Group Relative Policy Optimization. It does not divide advantages by the group’s reward standard deviation. It also sums token losses and divides by a constant response limit, here 3000, instead of dividing each response’s loss by its own length. These are the method’s two normalization changes [1,2]. Its policy loss uses a clipped probability ratio:

$$q_{it} = \frac{\pi_{\theta}(y_{it} \mid x_i, y_{i,<t})}{\pi_{\text{old}}(y_{it} \mid x_i, y_{i,<t})}, \quad L = -\frac{1}{B \, 3000} \sum_{i=1}^B \sum_{t \in \text{response}_i} \min(q_{it} A_i, \text{clip}(q_{it}, 0.8, 1.2) A_i).$$

Here the policy is the model’s next-token distribution, θ is its set of weights, and $B = 128$ responses. The group advantage is shared across all response tokens; this experiment does not determine which individual reasoning step deserves credit. We use zero KL penalty: no extra penalty keeps the updated policy close to a reference model.

How the single-GPU run fits together

One collection contains 16 questions times eight responses: 128 trajectories. The learner processes one trajectory at a time, accumulates 128 gradients, and takes one optimizer update. It then sends the new weights to vLLM for the next collection. We use one pass over each collection, full-model BF16 training, learning rate 10^{-6} , and the author’s Adam settings.

The pilot’s 64 questions gave four updates; the completed 512-question run gave 32 updates from 4096 sampled responses. Training and generation alternate on one A100 with 40 GB of memory. Putting the generator to sleep frees memory; it does not reset the weights. Likewise, switching a model between training and evaluation modes changes its execution behavior, not its learned weights. The optimizer performs the update.

Freeze the questions and the evaluator

We use the official code at commit `dfca49dd460e`, Oat 0.1.3.post1, vLLM 0.8.4, and Qwen model revision `4a83ca6e4526`. Full identifiers, package versions, and content hashes are recorded beside this guide in the research record and data manifest [4].

Purpose	Size	Selection and use
Pilot training	64	First 64 indices after shuffling 12,000 author training rows with Python <code>random.Random(42)</code> .
Larger training	512	First 512 indices from the same shuffle. This includes the pilot questions, but training restarts from the base weights.
Monitoring	64	Shuffle the author’s 500 MATH evaluation rows with seed 142; take the first 64. Reused for progress checks.
Final comparison	128	Take the next 128 from that evaluation shuffle. Disjoint from monitoring; used for the larger run’s fixed final comparison.

The 12,000 training rows include 4,501 IDs from the original MATH test split. An arbitrary download of that test split would therefore be a poor choice of evaluation data. We checked the author’s supplied training collection against the supplied MATH500 questions and found no exact overlaps after whitespace normalization. This does not rule out paraphrases or exposure during the base model’s pretraining.

Keep three kinds of settings separate

Weights. The baseline uses the original base checkpoint. The trained model uses the final saved checkpoint from the specified run. The 512-question run does not continue from the pilot’s weights.

Generation. Before and after evaluation use the same Qwen-Math prompt, which requests step-by-step reasoning and a boxed answer, the same 3000-token response limit, and greedy decoding with one answer per question. Training uses temperature 1 to obtain varied samples. Comparing a base model under a poorly matched prompt with a trained model under another prompt would mix prompt effects with weight changes. This is a central warning of the paper [1].

Scoring. The author’s training program uses a faster checker. The standalone evaluator uses an additional mathematical-equivalence check (`fast=False`). We apply the same standalone checker to both saved answer sets for the reported comparisons. The main fixed-test counts are 41 and 80. For the pilot, standalone counts were 20 and 22, whereas native training-checker counts were 19 and 21. We never combine scores from different checkers.

Choose the checkpoint before reading the final score

We retain the prescribed final checkpoint and evaluation, even if an earlier evaluation looks better. On the 64 monitoring questions, the main run’s final evaluation scored 36, versus 20 at baseline; another evaluation after the same 32 updates scored 39. The pilot likewise scored 23 and then 22 with identical saved weights. Greedy GPU decoding was not exactly repeatable. We report this variation without assigning it a specific numerical cause or choosing the higher score.

Learner seed 42 is fixed, but the unchanged author actor chooses a time-based generation seed. The run is therefore not fully deterministic. The large fixed-test gain is meaningful evidence for this run; evaluation repeats and independent training runs are still needed to assess its reproducibility.

More correct answers, fewer unfinished responses

The 512-question run restarted from the original Qwen2.5-Math-1.5B base weights. It completed one training epoch: 4096 sampled responses and 32 optimizer updates. We evaluated the prescribed final checkpoint, `step_00033`, on all 128 fixed test questions. Every prompt and reference answer matched the frozen manifest, and both answer sets were checked with the author’s full checker [4].

Fixed 128-question comparison	Base model	Final trained model
Correct final answers	41/128	80/128
Accuracy	32.0%	62.5%
Responses at the 3000-token cap	56/128	12/128

Paired question-by-question, 36 answers stayed correct, 43 stayed incorrect, 44 became correct, and five became incorrect. The net gain is 39/128, or **30.5 percentage points**. These test questions were separate from the 64 questions repeatedly used for monitoring. We have observed a substantial improvement on this fixed comparison, beyond establishing that the software can perform an optimizer update.

Completion is a material part of the result

Among the 44 newly correct answers, **36 had a baseline response that reached the token cap**, and **37 had no parseable baseline answer**. These categories can overlap; their counts must not be added. Seven gains came from baseline responses that did contain a parseable but incorrect answer.

These observations describe where gains occurred. They do not establish why training helped. A missing final answer may reflect repetition, an unfinished derivation, a wrong reasoning path, or an answer-format problem. Producing a correct completed response can involve changes in several of these behaviors at once. The evidence supports more useful answers under this protocol; it does not establish that all gains are newly learned mathematics or that adding answer formatting alone explains them.

What this result supports, and what remains open

The authors’ RL recipe can change this base model so that it returns many more correct answers on our fixed test questions. The training receipt records 396 mixed-reward groups out of 512, with 3168 nonzero advantages, and no saved token/reward alignment errors [4]. Reward, optimization, saved weights, and evaluation now have a concrete working reference.

This remains one training run at reduced scale. It does not isolate the two Dr. GRPO normalization changes from other training choices, reproduce the paper’s headline score, or validate credit assignment across RLM tool calls. The original question about RLM training therefore becomes more specific: can we preserve this verified update path while adding real tool actions, their results, and the correct reward boundaries?

The next comparisons

Full-500 comparisons and raw-question comparisons without the Qwen-Math prompt template are queued or running. Their results are pending. The full set contains the original 128 test questions, the 64 monitoring questions, and 308 additional questions; it expands coverage but is not a new independent training run. The raw-question comparisons will help establish how much the observed behavior depends on the prompt template. Both base and trained models must use the same settings within each comparison.

What did the pilot’s two extra points mean?

Under the standalone checker, 19 monitoring answers stayed correct, 41 stayed incorrect, three became correct, and one became incorrect. Thus the net change is $+2/64$, or 3.125 percentage points. These are the earlier pilot’s **monitoring results**, separate from the larger run’s 41-to-80 test result.

Changed problem	What the saved responses show
Fuel use	Repetition without an answer became a completed calculation and boxed 550.
Magic square	The baseline already derived $n = 7$, then repeated an incorrect verification. The final response completed and boxed 7.
Recurrence	A boxed answer changed from 1089 to the correct 898. Both responses had valid answer formatting.
Trigonometry	A correct boxed 12 became repetitive noncompletion at the 3000-token limit.

A concrete example: getting an answer out reliably

Karla drove 12,000 miles. How much fuel would she save with a car averaging 48 miles per gallon instead of 15? The completed post-training response gives

$$\frac{12000}{15} = 800, \quad \frac{12000}{48} = 250, \quad 800 - 250 = \boxed{550}.$$

The baseline repeated a version of the question until the 3000-token limit. The final response completed in approximately 417 text tokens. This is useful behavioral improvement on that example, but it does not show that the model newly acquired division or subtraction.

The recurrence example is also instructive. Independent calculation gives $211 + 420 + 267 = 898$. Both responses wrote Python describing the correct recurrence, but the baseline’s claimed code output was 1089. Those fenced “output” blocks were generated text; no Python tool had run. A correct final answer does not validate an entire explanation or prove successful tool use.

What the training evidence adds

The first collection had 12 rewarded responses out of 128, with seven mixed groups giving 56 nonzero advantages. Twenty responses reached the token limit. The four completed optimizer updates had finite nonzero gradients, sampled checkpoint tensors changed, and updated weights were sent back to the generator. This establishes an operational path from sampled answers to weight updates. It does not make the small monitoring gain statistically conclusive.

The pilot illustrates why an aggregate score needs accompanying examples. The larger run gives stronger evidence of improved answers, but its changed responses still need the same care in interpretation.

Sources and editable evidence.

[1] Liu et al., Understanding R1-Zero-Like Training: A Critical Perspective. <https://arxiv.org/abs/2503.20783>.

[2] Pinned official training, evaluation, and grading code; Oat package version.

[3] Pinned Qwen2.5-Math-1.5B model revision.

[4] Companion research record, data manifest, and resolved environment; main training receipt; pilot scoring receipt. The research record links the main fixed-test answer files and scoring evidence.